

PotatoGuard

GitHub Copilot Development Guide

Individual Prompts + Branch Instructions for Every Team Member

GitHub Repo	<code>https://github.com/SsemuliJoseph/Smart-Crop</code>
Project	<code>PotatoGuard - Irish Potato Disease Detection</code>
AI Tool	<code>GitHub Copilot Chat inside VS Code</code>
Group	<code>Group 7 BSc Computer Science 2024/2025</code>
Total Branches	<code>6 feature branches - one per team member</code>

HOW THIS DOCUMENT WORKS - READ THIS FIRST!

This document gives each team member their **OWN** personalised GitHub Copilot prompt. Use it inside VS Code to generate your assigned code automatically. Each person works on a **SEPARATE** branch so the lecturer can track every individual contribution on GitHub.

A	Find YOUR section in this document (search for your name). Each member gets their own colour-coded page.
B	Install VS Code and the GitHub Copilot extension. Clone the repository. Create YOUR branch using the exact name shown.
C	Open GitHub Copilot Chat (Ctrl+Shift+I). Copy the purple prompt box. Paste it into Copilot Chat and press Enter.
D	Accept the generated code. Save files in the correct folders. Then push to GitHub using the 3 git commands shown.
E	The lecturer will see every branch on GitHub and can check the CI test results (green = passing, red = failing).
F	NEVER work on someone else branch. After all branches are done, Team Lead (Ssemuli) merges everything into main.

Team Member	Branch Name (exact)	Role
Ssemuli Joseph	<code>feature/project-setup</code>	Team Lead & Project Manager
Musiimenta Dedicate	<code>feature/frontend</code>	Frontend Developer
Ayikoru Jackline	<code>feature/ai-model</code>	AI / ML Engineer
Tukahirwa Clinton	<code>feature/backend</code>	Backend Developer
Namata Juliet	<code>feature/database</code>	Database & Storage Engineer
Katusiime Moreen	<code>feature/testing</code>	Testing, Docs & DevOps

Ssemuli Joseph

2024/BCS/152/PS

Team Lead & Project Manager

Your GitHub Branch:

feature/project-setup

YOUR JOB IN THIS PROJECT:

You set up the ENTIRE project structure - the main app entry point, home screen, navigation between screens, and the README that explains the whole project to the team and your lecturer.

FILES YOU WILL CREATE:

README.md

.gitignore

frontend/App.js

frontend/src/screens/HomeScreen.js

frontend/src/navigation/AppNavigator.js

STEP-BY-STEP INSTRUCTIONS:

Install VS Code + GitHub Copilot

- 1 Download VS Code at code.visualstudio.com. Install the "GitHub Copilot" extension from the Extensions panel (Ctrl+Shift+X). Sign in with your GitHub account.

Clone the repository

- 2 Open VS Code. Press Ctrl+` to open the terminal. Type: `git clone https://github.com/SsemuliJoseph/Smart-Crop.git` then press Enter. Then type: `cd Smart-Crop`

Create your branch

- 3 In the terminal type: `git checkout -b feature/project-setup` then press Enter. You are now on YOUR personal branch.

Open GitHub Copilot Chat

- 4 Press Ctrl+Shift+I (or click the chat bubble on the left sidebar). A chat panel opens on the right.

Paste and run the prompt

- 5 Copy the purple box below. Paste it into the Copilot Chat input and press Enter. Wait for Copilot to generate the code.

Save the files Copilot creates

- 6 For each file Copilot generates, create that file in the correct folder in VS Code and paste the code in. Save with Ctrl+S.

Push to GitHub

- 7 In the terminal run these 3 commands: `git add .` then `git commit -m "Setup: project structure and home screen by Ssemuli Joseph"` then `git push origin feature/project-setup`

PASTE THIS INTO GITHUB COPILOT CHAT IN VS CODE:

You are helping build PotatoGuard - an AI-powered Irish potato disease detection system. I am Ssemuli Joseph, Team Lead. I need to set up the project structure.
Tech stack: React Native with Expo (mobile app) + AWS backend.

Generate these files:

1. README.md - Professional project README with:
 - Title: PotatoGuard - AI-Powered Irish Potato Disease Detection
 - Description of what the app does (detect Late Blight, Early Blight, Bacterial Wilt, Healthy)
 - Tech stack: React Native/Expo, AWS Lambda, API Gateway, SageMaker CNN, DynamoDB, S3
 - Team: Group 7 - BSc Computer Science 2024/2025

- Branch structure table: feature/project-setup, feature/frontend, feature/ai-model, feature/backend, feature/database, feature/testing
- Setup instructions: how to clone, install, and run

2. .gitignore - for React Native + Python + Node.js (ignore node_modules, .env, __pycache__, .h5 files)

3. frontend/App.js - Entry point with React Navigation:

- Import and set up a Stack Navigator
- Register screens: HomeScreen, UploadScreen, ResultScreen, HistoryScreen, LoginScreen
- Wrap everything in NavigationContainer

4. frontend/src/screens/HomeScreen.js:

- App title "PotatoGuard" in large bold text
- Subtitle: "Detect potato diseases instantly with AI"
- Green button "Scan a Leaf Now" -> navigates to UploadScreen
- Purple button "View History" -> navigates to HistoryScreen
- Simple clean styling, green (#1B5E20) and purple (#4A148C) colors
- Add StyleSheet at the bottom

Use functional components with hooks. Add comments on every important section.
Make variable names clear so a beginner can read the code.

TIP: After Copilot generates the code:

If code is incomplete, type in Copilot Chat: "Continue - show the rest of the code"

If something is wrong, type: "Fix the error in [filename] - explain what you changed"

To understand the code: "Explain this code line by line in simple English"

Musiimenta Dedicate

2024/BCS/116/PS

Frontend Developer

Your GitHub Branch:

feature/frontend

YOUR JOB IN THIS PROJECT:

You build the screens farmers actually use: the photo upload page (take a picture of the potato leaf), the results page (shows the disease and treatment), and the history page (list of all past scans).

FILES YOU WILL CREATE:

frontend/src/screens/UploadScreen.js

frontend/src/screens/ResultScreen.js

frontend/src/screens/HistoryScreen.js

frontend/src/components/DiseaseCard.js

frontend/src/services/api.js

STEP-BY-STEP INSTRUCTIONS:

1 Install VS Code + GitHub Copilot

Download VS Code. Install "GitHub Copilot" extension. Sign in with GitHub.

2 Clone the repo

Terminal (Ctrl+`): `git clone https://github.com/SsemuliJoseph/Smart-Crop.git` then `cd Smart-Crop`

3 Create your branch

Type: `git checkout -b feature/frontend` then press Enter.

4 Install Expo dependencies

Type: `cd frontend` then `npm install` then `npm install expo-image-picker axios`

5 Open Copilot Chat

Press Ctrl+Shift+I. Paste the prompt from the purple box below.

6 Create files in correct folders

Make folders: `frontend/src/screens/` and `frontend/src/components/` and paste each file Copilot generates.

7 Push to GitHub

`git add .` then `git commit -m "Feature: upload, result, and history screens by Musiimenta Dedicate"` then `git push origin feature/frontend`

PASTE THIS INTO GITHUB COPILOT CHAT IN VS CODE:

You are helping build PotatoGuard - an AI-powered Irish potato disease detection system.

I am Musiimenta Dedicate, the Frontend Developer. App is React Native with Expo.

API base URL variable: `const API_URL = "https://YOUR_API_GATEWAY_URL/prod";` (placeholder for now)

Generate these 5 files:

1. frontend/src/screens/UploadScreen.js:

- Two buttons: "Take Photo" (camera) and "Choose from Gallery" (image picker)
- Use expo-image-picker for both options
- Show a preview of the selected photo in a box on screen
- "Detect Disease" button that:
 - a) Converts the image to base64
 - b) Calls the detectDisease() function from api.js
 - c) Shows a loading spinner (ActivityIndicator) while waiting
 - d) On success: navigates to ResultScreen passing the result as a route param
 - e) On error: shows an Alert with the error message

2. frontend/src/screens/ResultScreen.js:
 - Read route.params to get: disease, confidence, treatment
 - Show disease name in LARGE text (fontSize 28, bold)
 - Show confidence as a percentage (e.g. "Confidence: 94%")
 - Show a colored banner depending on disease:
Healthy = green background, Late Blight = red, Early Blight = orange, Bacterial Wilt = purple
 - Show "Treatment Advice" section with the treatment text
 - "Scan Another Leaf" button -> goes back to UploadScreen
3. frontend/src/screens/HistoryScreen.js:
 - On screen load (useEffect), call getHistory("farmer001") from api.js
 - Show a FlatList of detection records
 - Each row shows: disease name (bold), date, confidence %
 - If the list is empty, show centered text: "No scans yet. Scan your first leaf!"
 - Add a pull-to-refresh feature
4. frontend/src/components/DiseaseCard.js:
 - A reusable card component that takes props: disease, confidence, date
 - Shows the disease name with an appropriate emoji (use leaf emoji for Healthy, warning for blight diseases)
 - Styled with rounded corners and a subtle shadow
5. frontend/src/services/api.js:
 - async function detectDisease(imageUri):
reads the image file, converts to base64, posts to API_URL + /detect
request body: { image_base64: "...", farmer_id: "farmer001" }
returns the JSON response from the server
 - async function getHistory(farmerId):
calls GET API_URL + /history/ + farmerId, returns the list of records
 - Both functions use try/catch and throw clear error messages

Use useState and useEffect hooks. Add a comment above every function explaining what it does.

TIP: After Copilot generates the code:

If code is incomplete, type in Copilot Chat: "Continue - show the rest of the code"

If something is wrong, type: "Fix the error in [filename] - explain what you changed"

To understand the code: "Explain this code line by line in simple English"

Ayikoru Jackline

2024/BCS/002

AI / ML Engineer

Your GitHub Branch:

feature/ai-model

YOUR JOB IN THIS PROJECT:

You build the AI brain of PotatoGuard. You train the Convolutional Neural Network (CNN) that looks at a photo of a potato leaf and decides what disease it has. This is the most technically impressive part of the whole project.

FILES YOU WILL CREATE:

model/train.py

model/evaluate.py

model/deploy.py

model/requirements.txt

model/README_model.md

STEP-BY-STEP INSTRUCTIONS:

1 Install VS Code + GitHub Copilot + Python extension

Download VS Code. Install GitHub Copilot AND the Python extension from Extensions panel (Ctrl+Shift+X).

2 Clone the repo

Terminal: `git clone https://github.com/SsemuliJoseph/Smart-Crop.git` then `cd Smart-Crop`

3 Create your branch

Type: `git checkout -b feature/ai-model` then Enter.

4 Get the PlantVillage dataset

Go to [kaggle.com](https://www.kaggle.com) and search "PlantVillage potato". Download the dataset. You will get folders: Potato__Early_blight, Potato__Late_blight, Potato__healthy. Put them inside a model/data/ folder.

5 Open Copilot Chat

Press Ctrl+Shift+I. Paste the prompt from the purple box below.

6 Run training in Google Colab (free GPU)

Go to colab.research.google.com. Upload train.py. Upload your dataset to Google Drive. Run the training there - it is much faster than your laptop.

7 Push to GitHub

`git add .` then `git commit -m "Feature: CNN model training and deployment scripts by Ayikoru Jackline"` then `git push origin feature/ai-model`

PASTE THIS INTO GITHUB COPILOT CHAT IN VS CODE:

You are helping build PotatoGuard - an AI-powered Irish potato disease detection system. I am Ayikoru Jackline, the AI/ML Engineer. I need Python scripts to train a CNN model.

Disease classes (4 total):

```
Index 0 = Healthy
Index 1 = Late Blight (Phytophthora infestans)
Index 2 = Early Blight (Alternaria solani)
Index 3 = Bacterial Wilt (Ralstonia solanacearum)
```

Dataset location after download from Kaggle:

```
model/data/Potato__healthy/
model/data/Potato__Early_blight/
model/data/Potato__Late_blight/
(Use 3 classes if Bacterial Wilt folder is not available in the dataset)
```

Generate these 5 files:

1. model/train.py - CNN training script using TensorFlow/Keras:
 - Use MobileNetV2 as the base model with imagenet weights (transfer learning)
 - Freeze the base model layers, add: GlobalAveragePooling2D, Dense(128, relu), Dropout(0.3), Dense(num_classes, softmax)
 - ImageDataGenerator with: rotation_range=20, width_shift=0.2, height_shift=0.2, horizontal_flip=True, zoom_range=0.2
 - Image size: 224x224, batch size: 32
 - Split: 80% train, 10% validation, 10% test
 - Train for 25 epochs with EarlyStopping(patience=5) and ModelCheckpoint to save best model
 - Save final model as model/potatoguard_model.h5
 - At the end print: "Training complete. Validation accuracy: X%"
 - We need validation accuracy >= 90%
2. model/evaluate.py - Evaluation script:
 - Load model from model/potatoguard_model.h5
 - Run predictions on test set
 - Print accuracy for each disease class separately
 - Print overall accuracy, precision, recall, F1 score using sklearn
 - Save a confusion matrix image to model/confusion_matrix.png using seaborn
 - Print PASS if accuracy >= 90%, FAIL with instructions if below
3. model/deploy.py - SageMaker deployment script:
 - Convert saved .h5 model to TensorFlow SavedModel format
 - Upload model to S3 bucket: potatoguard-models
 - Create SageMaker model and deploy real-time inference endpoint
 - Endpoint name: potatoguard-endpoint
 - Include test_single_image(image_path) function to test the endpoint with one photo
 - Print the prediction result with disease name and confidence score
4. model/requirements.txt:
tensorflow==2.12.0, numpy, Pillow, boto3, sagemaker, scikit-learn, matplotlib, seaborn
5. model/README_model.md:
 - Step 1: How to download PlantVillage dataset from Kaggle
 - Step 2: How to run train.py (both locally and in Google Colab)
 - Step 3: How to check if accuracy is >= 90% using evaluate.py
 - Step 4: How to deploy to SageMaker using deploy.py
 - Troubleshooting: what to do if accuracy is below 90%

Add a comment above EVERY function and every major code block explaining what it does.
A beginner should be able to follow the code step by step.

TIP: After Copilot generates the code:

If code is incomplete, type in Copilot Chat: "Continue - show the rest of the code"

If something is wrong, type: "Fix the error in [filename] - explain what you changed"

To understand the code: "Explain this code line by line in simple English"

Tukahirwa Clinton

2024/BCS/159/PS

Backend Developer

Your GitHub Branch:
feature/backend

YOUR JOB IN THIS PROJECT:

You build the server-side logic - the AWS Lambda functions that receive leaf photos from the app, call the AI model for a diagnosis, and send the result back to the farmer. You are the connection between the frontend app and the AI model.

FILES YOU WILL CREATE:

backend/detect/handler.py

backend/history/handler.py

backend/report/handler.py

backend/template.yaml

backend/README_backend.md

STEP-BY-STEP INSTRUCTIONS:

- 1 Install VS Code + GitHub Copilot**
Download VS Code. Install GitHub Copilot extension. Sign in.
- 2 Clone the repo**
Terminal: `git clone https://github.com/SsemuliJoseph/Smart-Crop.git` then `cd Smart-Crop`
- 3 Create your branch**
Type: `git checkout -b feature/backend` then Enter.
- 4 Install AWS SAM CLI**
For testing Lambda locally. Download from docs.aws.amazon.com/serverless-application-model. Run `sam --version` to confirm it is installed.
- 5 Open Copilot Chat**
Press `Ctrl+Shift+I`. Paste the purple prompt below.
- 6 Test locally**
After creating handler.py files, create a test event JSON and run: `sam local invoke DetectFunction --event test_event.json`
- 7 Push to GitHub**
`git add .` then `git commit -m "Feature: Lambda backend functions by Tukahirwa Clinton"` then `git push origin feature/backend`

PASTE THIS INTO GITHUB COPILOT CHAT IN VS CODE:

You are helping build PotatoGuard - an AI-powered Irish potato disease detection system. I am Tukahirwa Clinton, the Backend Developer. I need AWS Lambda functions in Python.

AWS resources used by this project:

SageMaker endpoint name: `potatoguard-endpoint`
DynamoDB table name: `potatoguard-detections`
S3 bucket for images: `potatoguard-images`
AWS Region: `us-east-1`

Disease treatment advice (hardcode these in the detect function):

Healthy: "Your plant is healthy! Keep monitoring weekly."
Late Blight: "Apply fungicide (Mancozeb or Chlorothalonil) immediately. Remove infected leaves."
Early Blight: "Apply copper-based fungicide. Improve air circulation between plants."

Bacterial Wilt: "Remove and destroy infected plants. Do not replant in same soil for 2 seasons."

Generate these 5 files:

1. backend/detect/handler.py - Main detection Lambda:

Event from API Gateway POST /detect:

```
body = { "image_base64": "...base64 string...", "farmer_id": "farmer001" }
```

The function must do these steps IN ORDER:

Step 1: Parse and validate the request body - return 400 if missing fields

Step 2: Decode the base64 image to bytes

Step 3: Generate a unique filename using uuid4()

Step 4: Upload image bytes to S3 bucket potatoguard-images

Step 5: Pre-process the image for the CNN: resize to 224x224, normalize to 0-1 range, expand dimensions to shape (1, 224, 224, 3)

Step 6: Invoke SageMaker endpoint potatoguard-endpoint with the processed image

Step 7: Parse SageMaker response - get predicted class index and confidence score

Step 8: Map index to disease name and get treatment text

Step 9: Generate the S3 image URL

Step 10: Save record to DynamoDB with all fields including timestamp

Step 11: Return JSON: { disease, confidence, treatment, imageUrl, detectionId, farmerId }

Add CORS headers. Return 500 with error message if any step fails.

2. backend/history/handler.py - History Lambda:

Event: GET /history/{farmerId}

- Query DynamoDB using farmerId-index Global Secondary Index

- Sort results by timestamp descending (newest first)

- Return list of records. Return empty list (not error) if no records found.

3. backend/report/handler.py - Single report Lambda:

Event: GET /report/{detectionId}

- Get item from DynamoDB by detectionId primary key

- Return full record including all fields

- Return 404 JSON if detectionId not found

4. backend/template.yaml - AWS SAM infrastructure template:

- Define 3 Lambda functions: DetectFunction, HistoryFunction, ReportFunction

- Set up API Gateway HTTP routes: POST /detect, GET /history/{farmerId}, GET

/report/{detectionId}

- Environment variables for each function: SAGEMAKER_ENDPOINT, DYNAMODB_TABLE, S3_BUCKET, AWS_REGION_NAME

- Lambda timeout: 30 seconds. Memory: 512MB

5. backend/README_backend.md - Step by step instructions:

- How to deploy: sam build then sam deploy --guided

- How to test locally with sam local invoke

- How to test API endpoints with Postman (with example request body)

- List of all required environment variables

Add a comment above every function and every major step inside each function.

TIP: After Copilot generates the code:

If code is incomplete, type in Copilot Chat: "Continue - show the rest of the code"

If something is wrong, type: "Fix the error in [filename] - explain what you changed"

To understand the code: "Explain this code line by line in simple English"

Namata Juliet

2024/BCS/127/PS

Database & Storage Engineer

Your GitHub Branch:

feature/database

YOUR JOB IN THIS PROJECT:

You set up and manage all the data storage for PotatoGuard: the database that permanently stores every disease detection result, and the cloud storage bucket that holds every leaf photo uploaded by farmers.

FILES YOU WILL CREATE:

database/dynamo_schema.json

database/setup_aws.py

database/s3_helper.py

database/dynamo_helper.py

database/README_database.md

STEP-BY-STEP INSTRUCTIONS:

1 Install VS Code + GitHub Copilot + Python

Download VS Code. Install GitHub Copilot. Install the Python VS Code extension. Make sure Python is installed on your computer.

2 Clone the repo

Terminal: `git clone https://github.com/SsemuliJoseph/Smart-Crop.git` then `cd Smart-Crop`

3 Create your branch

Type: `git checkout -b feature/database` then Enter.

4 Install Python packages

Type: `pip install boto3` to install the AWS Python library.

5 Set up AWS credentials

Type: `aws configure` in the terminal. Enter your AWS Access Key ID and Secret Access Key (get these from AWS Console > IAM > Users > Security credentials).

6 Open Copilot Chat

Press `Ctrl+Shift+I`. Paste the prompt from the purple box below.

7 Push to GitHub

`git add .` then `git commit -m "Feature: DynamoDB schema and S3 storage helpers by Namata Juliet"` then `git push origin feature/database`

PASTE THIS INTO GITHUB COPILOT CHAT IN VS CODE:

You are helping build PotatoGuard - an AI-powered Irish potato disease detection system. I am Namata Juliet, the Database and Storage Engineer. I use Python and AWS.

AWS resources I am setting up:

DynamoDB table: `potatoguard-detections`

S3 bucket for leaf images: `potatoguard-images`

S3 bucket for the AI model: `potatoguard-models`

AWS Region: `us-east-1`

Generate these 5 files:

1. `database/dynamo_schema.json` - JSON document describing the database design:

```
{
  "TableName": "potatoguard-detections",
  "PartitionKey": "detectionId" (String),
```

```

"SortKey": "timestamp" (String),
"GlobalSecondaryIndex": "farmerId-index" on farmerId field,
"AllAttributes": { detectionId, farmerId, imageUrl, imageS3Key,
                    disease, confidence, treatment, timestamp },
"BillingMode": "PAY_PER_REQUEST"
}

```

2. database/setup_aws.py - Script that creates all AWS resources:

- Create DynamoDB table with schema above (check if already exists first)
- Create S3 bucket potatoguard-images with CORS config for mobile app uploads
- Create S3 bucket potatoguard-models for the AI model files
- Add S3 bucket policy allowing Lambda to read and write images
- Print a clear success message after EACH resource is created
- At the end print: "Setup complete! All AWS resources created successfully."

3. database/s3_helper.py - Reusable S3 helper functions:

- upload_image(image_bytes, filename) -> returns the public S3 URL of the uploaded image
- generate_presigned_url(s3_key, expiry_seconds=3600) -> returns a temporary download URL
- delete_image(s3_key) -> deletes the image, returns True if successful
- list_farmer_images(farmer_id) -> returns list of all S3 keys for that farmer

Each function must have:

- A docstring explaining: what it does, what arguments it takes, what it returns
- try/except error handling with a helpful print message

4. database/dynamo_helper.py - Reusable DynamoDB helper functions:

- save_detection(detection_dict) -> saves a new record, returns the detectionId
- get_detection(detection_id) -> returns a single record or None if not found
- get_farmer_history(farmer_id) -> returns all records for a farmer, newest first
- delete_detection(detection_id) -> deletes a record, returns True if successful

Each function must have a docstring and try/except with a print on error.

5. database/README_database.md:

- Step 1: Install requirements: pip install boto3
- Step 2: Configure AWS credentials with aws configure
- Step 3: Run python setup_aws.py to create all resources
- Step 4: How to verify resources in AWS Console (with screenshots guide)
- Step 5: How other team members import and use s3_helper.py and dynamo_helper.py
- Common errors and fixes (access denied, bucket already exists, etc.)

Add detailed docstrings and comments everywhere. A complete beginner should understand the code.

TIP: After Copilot generates the code:

If code is incomplete, type in Copilot Chat: "Continue - show the rest of the code"

If something is wrong, type: "Fix the error in [filename] - explain what you changed"

To understand the code: "Explain this code line by line in simple English"

YOUR JOB IN THIS PROJECT:

You make sure the whole system works correctly. You write automated tests that check the backend and database, set up a GitHub CI pipeline so the lecturer sees green checkmarks when code is correct, and write the API documentation.

FILES YOU WILL CREATE:

```
tests/test_detect.py
tests/test_history.py
tests/test_model.py
tests/test_database.py
.github/workflows/ci.yml
docs/API_DOCUMENTATION.md
```

STEP-BY-STEP INSTRUCTIONS:

- 1 Install VS Code + GitHub Copilot**
Download VS Code. Install GitHub Copilot. Sign in.
- 2 Clone the repo**
Terminal: `git clone https://github.com/SsemuliJoseph/Smart-Crop.git` then `cd Smart-Crop`
- 3 Create your branch**
Type: `git checkout -b feature/testing` then Enter.
- 4 Install testing libraries**
Type: `pip install pytest pytest-cov moto boto3 requests`
- 5 Open Copilot Chat**
Press `Ctrl+Shift+I`. Paste the prompt from the purple box below.
- 6 Run tests locally**
After files are created, run: `python -m pytest tests/ -v` to see all tests pass (green) or fail (red).
- 7 Push to GitHub**
`git add .` then `git commit -m "Feature: test suite and CI pipeline by Katusiime Moreen"` then `git push origin feature/testing`

PASTE THIS INTO GITHUB COPILOT CHAT IN VS CODE:

You are helping build PotatoGuard - an AI-powered Irish potato disease detection system.
I am Katusiime Moreen, responsible for Testing, Documentation, and DevOps.

Project file locations:

```
backend/detect/handler.py - Lambda for disease detection (function name: lambda_handler)
backend/history/handler.py - Lambda for history
database/dynamo_helper.py - DynamoDB helper functions
database/s3_helper.py - S3 helper functions
```

AWS resource names:

```
DynamoDB table: potatoguard-detections
S3 bucket: potatoguard-images
SageMaker endpoint: potatoguard-endpoint
```

Generate these 6 files:

1. tests/test_detect.py - Tests for the detect Lambda using pytest + moto:
 - Use @mock_s3 and @mock_dynamodb decorators from moto to fake AWS (no real AWS needed)
 - test_detect_returns_200_for_valid_input():
Create a small valid 224x224 RGB base64 image, call lambda_handler, assert status code is 200, assert response body has: disease, confidence, treatment
 - test_detect_returns_400_if_image_missing():
Call with body missing image_base64 field, assert status 400
 - test_detect_returns_400_if_farmer_id_missing():
Call with body missing farmer_id, assert status 400
 - test_confidence_is_between_0_and_1():
Call with valid input, assert $0.0 \leq \text{confidence} \leq 1.0$
 - test_disease_is_valid_category():
Assert returned disease is one of: Healthy, Late Blight, Early Blight, Bacterial WiltAdd a clear docstring on EACH test explaining exactly what it checks.
2. tests/test_history.py - Tests for the history Lambda:
 - test_empty_history_returns_200_and_empty_list(): new farmer with no records -> 200, []
 - test_history_returns_correct_records(): save 2 records, call history, assert both returned
 - test_history_sorted_newest_first(): assert records are in descending timestamp order
3. tests/test_model.py - Model accuracy validation:
 - Skip gracefully if model/potatoguard_model.h5 does not exist yet
 - Load model and run on 20 test images (5 per disease class from model/data/)
 - test_model_meets_90_percent_accuracy(): assert overall accuracy ≥ 0.90
 - Print: "Model accuracy: X% - PASS" or "Model accuracy: X% - FAIL (retrain needed)"
4. tests/test_database.py - Tests for database helper functions:
 - Use moto to mock S3 and DynamoDB throughout
 - test_save_and_retrieve_detection(): save a record, get it back, check all fields match
 - test_get_farmer_history_returns_all_records(): save 3 records for same farmer, get all 3 back
 - test_upload_image_returns_url(): upload image bytes to mocked S3, assert URL is returned
 - test_delete_detection_removes_record(): save then delete, confirm get returns None
5. .github/workflows/ci.yml - GitHub Actions CI/CD pipeline:

```
name: PotatoGuard CI Tests
Trigger: on push to any feature/* branch AND on pull_request to main
Jobs:
  - Install Python 3.9
  - Install: pip install pytest pytest-cov moto boto3 tensorflow pillow scikit-learn
  - Run: python -m pytest tests/ -v --cov=backend --cov=database --cov-report=term
  - If any test fails: the job FAILS and shows which test failed (visible to lecturer)
  - If all tests pass: the job shows a green checkmark on the branch in GitHub
```
6. docs/API_DOCUMENTATION.md:

Document each of the 3 endpoints with:

 - Endpoint URL and HTTP method
 - Request example (with example JSON body)
 - Successful response example (with all fields explained)
 - Error response examples (400, 404, 500)

Include a "Testing with Postman" section with step by step instructions.

Include a "Common Errors and Solutions" table.

Write test names as plain English sentences so the lecturer can read what is being tested.
Add print statements in tests so team members can see progress when running them.

TIP: After Copilot generates the code:

If code is incomplete, type in Copilot Chat: "Continue - show the rest of the code"

If something is wrong, type: "Fix the error in [filename] - explain what you changed"

To understand the code: "Explain this code line by line in simple English"

PROMPT 7: FOR SSEMULI JOSEPH (TEAM LEAD) - MERGING ALL BRANCHES INTO MAIN

After ALL 6 team members have pushed their feature branches to GitHub, the Team Lead runs these steps to merge everything into main. This is the final step the lecturer will review.

Confirm all 6 branches exist on GitHub

- 1 Go to github.com/SsemuliJoseph/Smart-Crop. Click the "branches" tab. Confirm you see ALL 6 feature branches before you start merging.

Create a Pull Request for each branch on GitHub

- 2 On GitHub click "Pull requests" > "New pull request". Set base=main and compare=feature/frontend. Click "Create pull request". Repeat for all 6 branches.

Review and merge each Pull Request

- 3 Click "Merge pull request" > "Confirm merge" for each one. If there are conflicts, the affected files are shown - choose which version to keep.

OR merge from your terminal directly

- 4 Run the git commands in the code block below. This is faster than doing it one by one on the website.

```
# 1. Make sure you are on the main branch
git checkout main
git pull origin main

# 2. Merge each team member branch one at a time
git merge feature/project-setup
git merge feature/frontend
git merge feature/ai-model
git merge feature/backend
git merge feature/database
git merge feature/testing

# 3. Push the final merged main to GitHub
git push origin main

# 4. Tag the final release so the lecturer can find it easily
git tag -a v1.0 -m "PotatoGuard v1.0 - Group 7 final submission"
git push origin v1.0
```

PASTE THIS INTO GITHUB COPILOT CHAT IN VS CODE:

You are helping with the final integration of PotatoGuard - an AI potato disease detection system.

I am Ssemuli Joseph, Team Lead. All 6 branches have been merged into main.

Please do the following integration tasks:

1. Check and fix any import errors between files. Specifically:
 - backend/detect/handler.py should be able to import from database/dynamo_helper.py and database/s3_helper.py. Fix the import paths if they are wrong.
2. Create frontend/package.json if missing. Include all required dependencies:
expo, react-native, @react-navigation/native, @react-navigation/stack, expo-image-picker, axios, react-native-screens, react-native-safe-area-context
3. Create a root-level .env.example file listing all environment variables the team needs to configure before running the project:
API_BASE_URL, SAGEMAKER_ENDPOINT, DYNAMODB_TABLE, S3_BUCKET, S3_BUCKET_MODELS, AWS_REGION, AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY

4. Update README.md to add:
 - A "Project Status" section showing which features are complete
 - Instructions for the lecturer to test the live demo
 - A "Team Contributions" table listing each member and their branch
 5. Verify .github/workflows/ci.yml runs against the merged codebase.
Fix any file path errors in import statements inside the test files.
- List every problem you find and show the exact fix for each one clearly.